
UNIT 13 TEXT PROCESSING

Structure

- 13.0 Introduction
- 13.1 Objectives
- 13.2 Abundance of Digitized Text
- 13.3 Pattern-Matching Algorithms
- 13.4 Dynamic Programming
- 13.5 Text Compression and the Greedy Method
- 13.6 Tries
- 13.7 Summary
- 13.8 Solutions/ Answer to Check your Progress
- 13.9 References / Further Reading

13.0 Introduction

Text processing has become one of the most important areas of computer science due to the exponential growth of digital text data. From search engines processing billions of web pages to DNA sequencing analyzing genetic codes, efficient text processing algorithms are fundamental to modern computing applications.

Text processing encompasses various computational problems including pattern matching, text compression, information retrieval, natural language processing, and bioinformatics. These applications require sophisticated algorithms that can handle large volumes of text data efficiently while providing accurate and fast results.

This unit explores fundamental text processing algorithms and data structures. We examine pattern-matching algorithms for finding occurrences of patterns within text, dynamic programming approaches for sequence alignment and edit distance calculations, compression techniques using greedy algorithms, and specialized data structures like tries for efficient string storage and retrieval.

The algorithms and techniques covered in this unit form the foundation for more advanced applications in search engines, data compression tools, spell checkers, DNA analysis software, and machine translation systems.

13.1 Objectives

After completing this unit, you will be able to:

- Understand the significance and challenges of processing large-scale digitized text
- Implement various pattern-matching algorithms including brute force, Boyer-Moore, and Knuth-Morris-Pratt
- Apply dynamic programming techniques to solve text processing problems like edit distance and longest common subsequence
- Design and implement text compression algorithms using greedy methods and Huffman coding
- Create and manipulate trie data structures for efficient string storage and pattern matching
- Analyze time and space complexity of different text processing algorithms
- Choose appropriate algorithms based on specific text processing requirements

- Apply text processing techniques to real-world problems in bioinformatics, information retrieval, and data compression

13.2 Abundance of Digitized Text

The amount of digitized text has grown exponentially over the past decades. Consider these statistics:

- Google processes over 8.5 billion searches per day
- Social media platforms generate petabytes of text daily
- DNA databases contain billions of nucleotide sequences
- Digital libraries store millions of books and documents

Growth of Digital Text Data:

Year	Estimated Digital Text (Petabytes)
2010	~1
2015	~10
2020	~100
2025	~1000 (projected)

CHALLENGES IN TEXT PROCESSING

Scale Challenges:

- **Volume:** Processing terabytes of text data
- **Velocity:** Real-time processing requirements
- **Variety:** Multiple languages, formats, and encodings

Algorithmic Challenges:

- **Efficiency:** $O(n)$ or $O(n \log n)$ algorithms preferred over $O(n^2)$
- **Memory:** Working with datasets larger than available RAM
- **Accuracy:** Handling noise, errors, and variations in text

TEXT REPRESENTATION AND ENCODING

Understanding text encoding is crucial for efficient processing:

Algorithm Explanation:

Text encoding is like having different languages for computers to understand human text. Just as humans use different alphabets (Latin, Cyrillic, Chinese), computers use different encoding schemes to represent text.

The Fundamental Problem: Computers only understand numbers (0s and 1s), but humans work with letters, symbols, and characters. We need a systematic way to convert between human text and computer numbers.

Historical Evolution of Encoding:

ASCII (American Standard Code for Information Interchange):

Purpose: The original encoding system for English text

Range: Uses 7 bits to represent 128 characters (0-127)

Coverage: English letters, digits, punctuation, and control characters

Limitation: Only works for English and basic symbols

Unicode Revolution:

Purpose: A universal encoding system for all world languages

Goal: Assign a unique number (code point) to every character in every language

Coverage: Over 1 million possible characters including emojis, ancient scripts, mathematical symbols

Common Unicode Encodings:

UTF-8 (8-bit Unicode Transformation Format):

Smart design: Uses 1-4 bytes per character depending on what's needed

ASCII compatible: First 128 characters match ASCII exactly

Variable length: Common characters use fewer bytes, rare ones use more

UTF-16 and UTF-32:

UTF-16: Uses 2 or 4 bytes per character

UTF-32: Uses exactly 4 bytes per character

Tradeoff: More predictable but potentially wasteful of space

Text Encoding Algorithm Steps:

Step 1: Character Analysis

Identify the character: Look at each character in the text

Determine code point: Find the Unicode number assigned to this character

Check character type: Is it ASCII, Latin-1, or requires full Unicode?

Step 2: Choose Encoding Scheme

Analyze text composition: What types of characters are present?

Consider space efficiency: How much storage is available?

Think about compatibility: What systems will read this text?

Step 3: Apply Encoding Rules

For UTF-8 encoding:

ASCII characters (0-127): Use 1 byte as-is

Extended characters (128-2047): Use 2 bytes

Most other characters: Use 3 bytes

Rare/special characters: Use 4 bytes

Step 4: Generate Byte Sequence

Convert each character: Apply the encoding rules to get bytes

Concatenate results: Combine all bytes into final encoded text

Add metadata if needed: Some formats include encoding information

Visual Encoding Diagrams:

ASCII Encoding Example:

Character: 'A'

ASCII Code: 65

Binary: 01000001

Bytes needed: 1

Storage: [01000001]

UTF-8 Encoding Examples:

Character: 'A' (ASCII)

Unicode: U+0041 (65)

UTF-8: 01000001

Bytes: [01000001] (1 byte)

Character: 'é' (Latin Extended)

Unicode: U+00E9 (233)

UTF-8: 11000011 10101001

Bytes: [11000011, 10101001] (2 bytes)

Character: '世' (Chinese)

Unicode: U+4E16 (19990)

UTF-8: 11100100 10111000 10010110

Bytes: [11100100, 10111000, 10010110] (3 bytes)

Practical Encoding Algorithm:

```

def analyze_text_encoding_needs(text):
    """
    Algorithm to determine optimal encoding for given text.

    Step-by-step process:
    1. Analyze character types in the text
    2. Calculate storage requirements for different encodings
    3. Recommend best encoding choice
    4. Show detailed breakdown of encoding process
    """

    print(f"=== Text Encoding Analysis for: '{text}' ===")

    # Step 1: Character Analysis
    char_analysis = {
        'ascii_count': 0, # Characters 0-127
        'latin1_count': 0, # Characters 128-255
        'unicode_bmp_count': 0, # Characters 256-65535 (Basic Multilingual Plane)
        'unicode_extended_count': 0, # Characters > 65535
        'total_chars': len(text)
    }

    print("\nStep 1: Character Analysis")
    print("Character | Unicode | Type | UTF-8 Bytes Needed")
    print("-" * 50)

    for char in text:
        code_point = ord(char)

        if code_point <= 127:
            char_type = "ASCII"
            char_analysis['ascii_count'] += 1
            utf8_bytes = 1
        elif code_point <= 255:
            char_type = "Latin-1"
            char_analysis['latin1_count'] += 1
            utf8_bytes = 2
        elif code_point <= 65535:
            char_type = "Unicode BMP"
            char_analysis['unicode_bmp_count'] += 1
            utf8_bytes = 3
        else:
            char_type = "Unicode Extended"
            char_analysis['unicode_extended_count'] += 1
            utf8_bytes = 4

    print(f"  {char}    | U+{code_point:04X} | {char_type:12} | {utf8_bytes}")

    # Step 2: Calculate storage requirements
    print(f"\nStep 2: Storage Requirements Calculation")

```

```

# ASCII encoding (only works if all characters are ASCII)
ascii_possible = char_analysis['latin1_count'] == 0 and \
char_analysis['unicode_bmp_count'] == 0 and \
char_analysis['unicode_extended_count'] == 0

ascii_bytes = len(text) if ascii_possible else "Not possible"

# UTF-8 encoding
utf8_bytes = (char_analysis['ascii_count'] * 1 +
char_analysis['latin1_count'] * 2 +
char_analysis['unicode_bmp_count'] * 3 +
char_analysis['unicode_extended_count'] * 4)

# UTF-16 encoding
utf16_bytes = (char_analysis['ascii_count'] * 2 +
char_analysis['latin1_count'] * 2 +
char_analysis['unicode_bmp_count'] * 2 +
char_analysis['unicode_extended_count'] * 4)

# UTF-32 encoding
utf32_bytes = len(text) * 4

print(f'ASCII: {ascii_bytes} bytes')
print(f'UTF-8: {utf8_bytes} bytes')
print(f'UTF-16: {utf16_bytes} bytes')
print(f'UTF-32: {utf32_bytes} bytes')

# Step 3: Encoding recommendation
print(f"\nStep 3: Encoding Recommendation")

if ascii_possible:
    recommended = "ASCII or UTF-8"
    reason = "All characters are ASCII-compatible"
elif char_analysis['unicode_bmp_count'] == 0 and
char_analysis['unicode_extended_count'] == 0:
    recommended = "UTF-8"
    reason = "Mix of ASCII and Latin characters - UTF-8 is most efficient"
elif utf8_bytes <= utf16_bytes:
    recommended = "UTF-8"
    reason = "UTF-8 provides better compression for this text"
else:
    recommended = "UTF-16"
    reason = "Many non-ASCII characters - UTF-16 might be more
efficient"

print(f'Recommended encoding: {recommended}')
print(f'Reason: {reason}')

return {
    'analysis': char_analysis,
    'storage_requirements': {
        'ascii': ascii_bytes,
        'utf8': utf8_bytes,

```

```

        'utf16': utf16_bytes,
        'utf32': utf32_bytes
    },
    'recommendation': recommended
}

def demonstrate_utf8_encoding_process(text):
    """
    Show step-by-step UTF-8 encoding process for each character.
    """
    print(f"\n=== UTF-8 Encoding Process for: '{text}' ===")

    result_bytes = []

    for i, char in enumerate(text):
        code_point = ord(char)
        print(f"\nCharacter {i+1}: '{char}'")
        print(f"Unicode code point: U+{code_point:04X} ({code_point})")

        if code_point <= 0x7F: # 1-byte sequence
            utf8_bytes = [code_point]
            print(f"Range: 0x00-0x7F (ASCII)")
            print(f"UTF-8 pattern: 0xxxxxxx")
            print(f"Binary: {code_point:08b}")
            print(f"UTF-8 bytes: [{code_point:02X}]")

        elif code_point <= 0x7FF: # 2-byte sequence
            byte1 = 0b11000000 | (code_point >> 6)
            byte2 = 0b10000000 | (code_point & 0b00111111)
            utf8_bytes = [byte1, byte2]
            print(f"Range: 0x80-0x7FF (Latin Extended)")
            print(f"UTF-8 pattern: 110xxxxx 10xxxxxx")
            print(f"Binary: {code_point:011b}")
            print(f"Split: {code_point >> 6:05b} + {code_point & 0b00111111:06b}")
            print(f"UTF-8 bytes: [{byte1:02X}, {byte2:02X}]")

        elif code_point <= 0xFFFF: # 3-byte sequence
            byte1 = 0b11100000 | (code_point >> 12)
            byte2 = 0b10000000 | ((code_point >> 6) & 0b00111111)
            byte3 = 0b10000000 | (code_point & 0b00111111)
            utf8_bytes = [byte1, byte2, byte3]
            print(f"Range: 0x800-0xFFFF (Basic Multilingual Plane)")
            print(f"UTF-8 pattern: 1110xxxx 10xxxxxx 10xxxxxx")
            print(f"Binary: {code_point:016b}")
            print(f"Split: {code_point >> 12:04b} + {(code_point >> 6) & 0b00111111:06b} + {code_point & 0b00111111:06b}")
            print(f"UTF-8 bytes: [{byte1:02X}, {byte2:02X}, {byte3:02X}]")

        else: # 4-byte sequence
            byte1 = 0b11110000 | (code_point >> 18)
            byte2 = 0b10000000 | ((code_point >> 12) & 0b00111111)
            byte3 = 0b10000000 | ((code_point >> 6) & 0b00111111)
            byte4 = 0b10000000 | (code_point & 0b00111111)
            utf8_bytes = [byte1, byte2, byte3, byte4]

```

```

print(f"Range: 0x10000+ (Extended Unicode)")
print(f"UTF-8 pattern: 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx")
print(f"UTF-8 bytes: [{byte1:02X}, {byte2:02X}, {byte3:02X},
{byte4:02X}]"")

result_bytes.extend(utf8_bytes)

print(f"\nFinal UTF-8 encoding: {[f'{b:02X}' for b in result_bytes]}")
print(f"Total bytes needed: {len(result_bytes)}")

    return result_bytes

def create_encoding_comparison_chart():
    """
    Create a visual comparison of different encoding schemes.
    """
    print("\n=== Encoding Comparison Chart ===")

    test_cases = [
        ("Hello", "English text"),
        ("Café", "French text with accents"),
        ("Привет", "Russian Cyrillic text"),
        ("こんにちは", "Japanese Hiragana"),
        ("🌍🚀📺", "Emojis"),
        ("Hello世界🌍", "Mixed multilingual")
    ]

    print(f'{"Text":<12} {"Description":<25} {"ASCII":<8} {"UTF-8":<8} {"UTF-16":<8} {"UTF-32":<8} {"UTF-32":<8}')
    print("-" * 75)

    for text, description in test_cases:
        try:
            ascii_size = len(text.encode('ascii'))
        except:
            ascii_size = "N/A"

        utf8_size = len(text.encode('utf-8'))
        utf16_size = len(text.encode('utf-16'))
        utf32_size = len(text.encode('utf-32'))

        print(f'{text:<12} {description:<25} {str(ascii_size):<8} {utf8_size:<8}
{utf16_size:<8} {utf32_size:<8}')

# Demonstration of text encoding concepts
def demonstrate_text_encoding():
    """Comprehensive demonstration of text encoding concepts."""
    print("=== COMPREHENSIVE TEXT ENCODING DEMONSTRATION ===")

# Test cases with different character types
test_texts = [
    "Hello", # Pure ASCII

```

```

    "Café au lait", # ASCII + Latin-1
    "Hello世界", # ASCII + Chinese
    "🌐🔗🖨️", # Emojis (4-byte UTF-8)
]

for text in test_texts:
print("\n" + "="*60)
    result = analyze_text_encoding_needs(text)
    demonstrate_utf8_encoding_process(text)

print("\n" + "="*60)
create_encoding_comparison_chart()

# Run the demonstration
demonstrate_text_encoding()
import sys
import unicodedata

class TextAnalyzer:
    """Class for analyzing text properties and encodings."""

    def __init__(self, text):
        """Initialize with text string."""
        self.text = text
        self.length = len(text)

    def analyze_encoding(self):
        """Analyze text encoding properties."""
        print(f'Text: '{self.text}''')
        print(f'Length: {self.length} characters')
        print(f'Memory size: {sys.getsizeof(self.text)} bytes')

# Character analysis
char_types = {
    'ascii': 0,
    'latin': 0,
    'unicode': 0,
    'digits': 0,
    'spaces': 0,
    'punctuation': 0
}

for char in self.text:
    if ord(char) < 128:
        char_types['ascii'] += 1
    elif ord(char) < 256:
        char_types['latin'] += 1
    else:
        char_types['unicode'] += 1

    if char.isdigit():
        char_types['digits'] += 1
    elif char.isspace():
        char_types['spaces'] += 1

```

```

elif not char.isalnum():
    char_types['punctuation'] += 1

print("Character distribution:")
for char_type, count in char_types.items():
    if count > 0:
        percentage = (count / self.length) * 100
print(f' {char_type}: {count} ({percentage:.1f}%)')

def get_encoding_sizes(self):
    """Compare different encoding sizes."""
    encodings = ['utf-8', 'utf-16', 'utf-32', 'ascii']
    sizes = {}

    for encoding in encodings:
        try:
            encoded = self.text.encode(encoding)
            sizes[encoding] = len(encoded)
        except UnicodeEncodeError:
            sizes[encoding] = "Cannot encode"

print("\nEncoding sizes (bytes):")
for encoding, size in sizes.items():
    print(f' {encoding}: {size}')

    return sizes

# Demonstrate text analysis
def demonstrate_text_analysis():
    """Demonstrate text analysis and encoding."""
    print("=== Text Analysis Demonstration ===")

# Sample texts in different languages
texts = [
    "Hello, World!",
    "Café au lait",
    "こんにちは世界",
    "DNA sequence: ATCGATCG",
    "Mixed: Hello 世界 123!"
]

for text in texts:
    print(f"\n--- Analysis for: '{text}' ---")
    analyzer = TextAnalyzer(text)
    analyzer.analyze_encoding()
    analyzer.get_encoding_sizes()

demonstrate_text_analysis()

```

APPLICATIONS OF TEXT PROCESSING

Search Engines:

- Web crawling and indexing
- Query processing and ranking

- Snippet generation

Bioinformatics:

- DNA/RNA sequence analysis
- Protein structure prediction
- Genome assembly

Natural Language Processing:

- Machine translation
- Sentiment analysis
- Named entity recognition

Data Compression:

- File compression utilities
- Database compression
- Network data transmission

13.3 Pattern-Matching Algorithms

Pattern matching is fundamental to text processing - finding all occurrences of a pattern string P within a text string T.

Pattern Matching Visualization:

Text: "ABAAABCDABABCABCABCDAB"

Pattern: "ABCD"

Matches: positions 5 and 17

T: A B A A A B C D A B A B C A B C A B C D A B

P: A B C D A B C D

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21

BRUTE FORCE ALGORITHM

The simplest approach checks every possible position:

Algorithm Explanation:

The brute force pattern matching algorithm works like reading a book and looking for a specific word:

1. **Start at the beginning:** Place your finger at the first character of the text
2. **Try to match:** Compare each character of the pattern with the text starting from your current position
3. **If all characters match:** You found the pattern! Record the position
4. **If any character doesn't match:** Move your finger one position forward in the text and try again
5. **Repeat:** Continue this process until you reach the end of the text

If you're looking for the word "HELLO" in a sentence. You start at the first letter, check if "H-E-L-L-O" matches the next 5 characters. If not, you move one letter forward and check again. This continues until you've checked every possible starting position.

Why is it called "brute force"?: Because it doesn't use any clever tricks - it simply tries every possibility, even when it's obvious that some positions can't work.

```
def brute_force_search(text, pattern):
    """
    Brute force pattern matching algorithm.
    Time Complexity: O(nm) where n = len(text), m = len(pattern)
    Space Complexity: O(1)
```

```

"""
n, m = len(text), len(pattern)
matches = []

# Check each possible starting position
for i in range(n - m + 1):
    # Check if pattern matches at position i
    match = True
    for j in range(m):
        if text[i + j] != pattern[j]:
            match = False
            break

    if match:
matches.append(i)

return matches

def brute_force_search_detailed(text, pattern):
    """Brute force with detailed step-by-step output."""
    n, m = len(text), len(pattern)
    matches = []
    comparisons = 0

    print(f'Searching for pattern '{pattern}' in text '{text}')
    print(f'Text length: {n}, Pattern length: {m}')
    print()

    for i in range(n - m + 1):
        print(f'Position {i}:')
        print(f'Text: {text}')
        print(f'Pattern: {' ' * i} {pattern}')

        match = True
        j = 0
        while j < m:
            comparisons += 1
            if text[i + j] != pattern[j]:
                match = False
        print(f'{' ' * (i + j)} ^ Mismatch at position {j}')
        break
        j += 1

        if match:
matches.append(i)
        print(f'{' ' * i} {'✓' * m} MATCH found!")

    print()

    print(f'Total comparisons: {comparisons}')
    print(f'Matches found at positions: {matches}')
    return matches

# Demonstrate brute force algorithm

```

```

def demonstrate_brute_force():
    """Demonstrate brute force pattern matching."""
    print("=== Brute Force Pattern Matching ===")

    test_cases = [
        ("ABAAABCDABABCABCABCDAB", "ABCD"),
        ("AAAAAAAAAAAA", "AAA"),
        ("HELLO WORLD", "WORLD"),
        ("ABCDEF", "XYZ")
    ]

    for text, pattern in test_cases:
        print(f"\n--- Test Case ---")
        matches = brute_force_search_detailed(text, pattern)
        print(f"Result: {len(matches)} matches found")
        print("-" * 50)

    demonstrate_brute_force()

```

KNUTH-MORRIS-PRATT (KMP) ALGORITHM

KMP algorithm avoids redundant comparisons by preprocessing the pattern to create a failure function:

Algorithm Explanation:

The KMP algorithm is like having a smart bookmark system when searching for a pattern:

Phase 1 - Preprocessing (Building the LPS Array):

1. **Analyze the pattern first:** Before searching, examine the pattern to find places where it repeats itself
2. **Create a "failure function":** For each position in the pattern, record how many characters you can skip if a mismatch occurs
3. **The key insight:** If you've already matched some characters and then find a mismatch, you don't need to start over completely - you can use previously matched characters

Phase 2 - Searching:

1. **Start comparing:** Compare text and pattern characters one by one
2. **When characters match:** Move forward in both text and pattern
3. **When mismatch occurs:** Instead of starting over, use the failure function to determine how far back to move in the pattern
4. **Never go backward in text:** This is the key efficiency - we never re-examine text characters

Real-world analogy: It's like proofreading a document. If you're checking for the word "ABABAB" and you've matched "ABAB" but the next character doesn't match, you don't start over. You realize that the "AB" you just matched could be the start of a new attempt, so you continue from there.

Why is it faster?: Because it eliminates redundant work by remembering what it has already learned about the pattern.

```

def compute_lps(pattern):
    """
    Compute Longest Prefix Suffix (LPS) array for KMP algorithm.
    LPS[i] = length of longest proper prefix which is also suffix for pattern[0:i+1]
    """
    m = len(pattern)

```

```

lps = [0] * m
length = 0 # length of previous longest prefix suffix
i = 1

print(f"Computing LPS array for pattern: '{pattern}'")
print(f"Initial LPS: {lps}")

while i < m:
    if pattern[i] == pattern[length]:
        length += 1
    lps[i] = length
    print(f"Match at {i}: pattern[{i}] = pattern[{length-1}], LPS[{i}] = {length}")
    i += 1
    else:
        if length != 0:
            length = lps[length - 1]
        print(f"Mismatch at {i}: backtrack length to {length}")
        else:
            lps[i] = 0
            print(f"No prefix match at {i}: LPS[{i}] = 0")
            i += 1

print(f"Final LPS array: {lps}")
return lps

def kmp_search(text, pattern):
    """
    Knuth-Morris-Pratt pattern matching algorithm.
    Time Complexity: O(n + m)
    Space Complexity: O(m)
    """
    n, m = len(text), len(pattern)
    if m == 0:
        return []

    # Preprocess pattern to get LPS array
    lps = compute_lps(pattern)

    matches = []
    i = 0 # index for text
    j = 0 # index for pattern
    comparisons = 0

    print(f"\nSearching using KMP algorithm:")
    print(f"Text: '{text}'")
    print(f"Pattern: '{pattern}'")
    print()

    while i < n:
        comparisons += 1
        print(f"Comparing text[{i}]='{text[i]}' with pattern[{j}]='{pattern[j]}'")

        if text[i] == pattern[j]:
            print(f" Match! Moving both pointers")

```

```

i += 1
    j += 1

    if j == m:
        # Found a match
        match_pos = i - j
        matches.append(match_pos)
        print(f" PATTERN FOUND at position {match_pos}!")
        j = lps[j - 1] # Get next position from LPS
        print(f" Using LPS: next j = {j}")
    elif i < n and text[i] != pattern[j]:
        # Mismatch after j matches
        if j != 0:
            j = lps[j - 1] # Don't match lps[0..lps[j-1]] chars
        print(f" Mismatch! Using LPS: j = lps[{j}] = {lps[j-1]}")
    else:
        print(f" Mismatch at start of pattern, advance text pointer")
    i += 1
print()

print(f"Total comparisons: {comparisons}")
print(f"Matches found at positions: {matches}")
return matches

# Demonstrate KMP algorithm
def demonstrate_kmp():
    """Demonstrate KMP pattern matching algorithm."""
    print("=== KMP Pattern Matching Algorithm ===")

    # Example showing KMP efficiency
    text = "ABABCABABA"
    pattern = "ABABA"

    print("First, let's compute the LPS array:")
    lps = compute_lps(pattern)

    print("\nNow, let's search for the pattern:")
    matches = kmp_search(text, pattern)

demonstrate_kmp()

```

13.4 Dynamic Programming

Dynamic programming is particularly useful for text processing problems involving sequence alignment, edit distance, and longest common subsequences.

EDIT DISTANCE (LEVENSHTEIN DISTANCE)

The edit distance between two strings is the minimum number of operations (insert, delete, substitute) needed to transform one string into another.

Algorithm Explanation:

The edit distance algorithm is like finding the cheapest way to transform one word into another using a limited set of operations:

The Problem: How do you change "CAT" into "DOG" with the minimum number of edits?

Available Operations:

1. **Insert:** Add a new character anywhere
2. **Delete:** Remove any character
3. **Substitute:** Replace one character with another

The Dynamic Programming Approach:

1. **Build a table:** Create a grid where rows represent characters of the first string and columns represent characters of the second string
2. **Fill base cases:** The first row and column represent the cost of transforming from/to an empty string (just insertions or deletions)
3. **Fill the table systematically:** For each cell, consider three possibilities:
 - **Characters match:** No operation needed, copy the diagonal value
 - **Characters don't match:** Take the minimum of:
 - Left cell + 1 (insert operation)
 - Top cell + 1 (delete operation)
 - Diagonal cell + 1 (substitute operation)

Real-world analogy: It's like finding the shortest path on a grid map where you can move right (insert), down (delete), or diagonally (substitute/match). Each move has a cost, and you want the cheapest total journey.

Why dynamic programming?: We break the problem into smaller subproblems. To transform "CAT" to "DOG", we need to know the cost of transforming "CA" to "DO", "CA" to "DOG", and "CAT" to "DO". We build up the solution from these smaller pieces.

Applications: Spell checkers, DNA sequence analysis, version control systems, plagiarism detection.

```
def edit_distance(str1, str2):
    """
    Compute edit distance using dynamic programming.
    Time Complexity: O(mn)
    Space Complexity: O(mn)
    """
    m, n = len(str1), len(str2)

    # Create DP table
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Initialize base cases
    for i in range(m + 1):
        dp[i][0] = i # Delete all characters from str1
    for j in range(n + 1):
        dp[0][j] = j # Insert all characters from str2

    print(f"Computing edit distance between '{str1}' and '{str2}'")
    print("\nDP Table construction:")

    # Fill DP table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if str1[i-1] == str2[j-1]:
                # Characters match, no operation needed
                dp[i][j] = dp[i-1][j-1]
            else:
                # Take minimum of three operations
```

```

dp[i][j] = 1 + min(
    dp[i-1][j], # Delete from str1
    dp[i][j-1], # Insert into str1
    dp[i-1][j-1] # Substitute
)

# Print DP table
print_dp_table(dp, str1, str2)

return dp[m][n]

def print_dp_table(dp, str1, str2):
    """Print DP table in formatted way."""
    m, n = len(str1), len(str2)

    # Header
    print(" ", end="")
    print("\\".rjust(3), end="")
    for char in str2:
        print(f"{char}" .rjust(3), end="")
    print()

    # Rows
    for i in range(m + 1):
        if i == 0:
            row_label = "\\"
        else:
            row_label = str1[i-1]
        print(f"{row_label}" .rjust(3), end=" ")

        for j in range(n + 1):
            print(f"{dp[i][j]}" .rjust(3), end="")
        print()

def edit_distance_with_operations(str1, str2):
    """
    Compute edit distance and track operations performed.
    Returns (distance, list of operations)
    """
    m, n = len(str1), len(str2)

    # DP table
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Operation tracking table
    ops = ["" * (n + 1) for _ in range(m + 1)]

    # Initialize base cases
    for i in range(m + 1):
        dp[i][0] = i
        ops[i][0] = "D" * i # Delete operations
    for j in range(n + 1):
        dp[0][j] = j
        ops[0][j] = "I" * j # Insert operations

```

```

# Fill tables
for i in range(1, m + 1):
    for j in range(1, n + 1):
        if str1[i-1] == str2[j-1]:
            dp[i][j] = dp[i-1][j-1]
            ops[i][j] = ops[i-1][j-1] + "M" # Match
        else:
            # Find minimum operation
            delete_cost = dp[i-1][j] + 1
            insert_cost = dp[i][j-1] + 1
            substitute_cost = dp[i-1][j-1] + 1

            min_cost = min(delete_cost, insert_cost, substitute_cost)
            dp[i][j] = min_cost

            if min_cost == delete_cost:
                ops[i][j] = ops[i-1][j] + "D"
            elif min_cost == insert_cost:
                ops[i][j] = ops[i][j-1] + "I"
            else:
                ops[i][j] = ops[i-1][j-1] + "S"

    return dp[m][n], ops[m][n]

def demonstrate_edit_distance():
    """Demonstrate edit distance calculation."""
    print("==== Edit Distance Demonstration ====")

    test_pairs = [
        ("CAT", "DOG"),
        ("KITTEN", "SITTING"),
        ("SUNDAY", "SATURDAY"),
        ("INTENTION", "EXECUTION")
    ]

    for str1, str2 in test_pairs:
        print(f"\n--- Edit Distance: '{str1}' → '{str2}' ---")
        distance = edit_distance(str1, str2)
        print(f"Edit distance: {distance}")

        distance_ops, operations = edit_distance_with_operations(str1, str2)
        print(f"Operations sequence: {operations}")
        print(f" M=Match, S=Substitute, I=Insert, D=Delete")

    demonstrate_edit_distance()

```

LONGEST COMMON SUBSEQUENCE (LCS)

Find the longest subsequence common to two sequences.

LCS Algorithm - High Level Steps

- **Step 1: Create DP Table**
- Build a 2D table of size $(m+1) \times (n+1)$ where m and n are the lengths of the two strings
- Initialize all cells to 0
- **Step 2: Fill the Table**

- For each cell (i,j), compare characters from both strings:
 - If characters match: take diagonal value + 1
 - If characters don't match: take maximum of left or top cell
- **Step 3: Extract Result**
- The bottom-right cell contains the LCS length
- To get the actual LCS string, backtrack from bottom-right to top-left, following the path that created the optimal solution
- **Step 4: Backtrack (for actual LCS string)**
- Start from (m,n) and work backwards
- When characters match: include character and move diagonally
- When they don't match: move toward the cell that provided the maximum value
- The algorithm builds up the solution incrementally by solving smaller subproblems and uses the results to construct the final answer.

Given below is the sample code implementation -

```
def lcs_length(str1, str2):
    """
    Compute length of Longest Common Subsequence using DP.
    Time Complexity: O(mn)
    Space Complexity: O(mn)
    """
    m, n = len(str1), len(str2)

    # Create DP table
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    print(f"Computing LCS length between '{str1}' and '{str2}'")

    # Fill DP table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if str1[i-1] == str2[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    # Print DP table
    print("\nLCS DP Table:")
    print_dp_table(dp, str1, str2)

    return dp[m][n]

def lcs_string(str1, str2):
    """
    Compute actual LCS string using DP with backtracking.
    """
    m, n = len(str1), len(str2)

    # Create DP table
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Fill DP table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
```

```

        if str1[i-1] == str2[j-1]:
dp[i][j] = dp[i-1][j-1] + 1
        else:
dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    # Backtrack to find LCS string
    lcs = []
i, j = m, n

print(f"\nBacktracking to find LCS:")
    while i > 0 and j > 0:
        if str1[i-1] == str2[j-1]:
lcs.append(str1[i-1])
print(f" Match '{str1[i-1]}' at ({i},{j}) - added to LCS")
i -= 1
            j -= 1
        elif dp[i-1][j] > dp[i][j-1]:
print(f" Move up from ({i},{j}) to ({i-1},{j})")
i -= 1
            else:
print(f" Move left from ({i},{j}) to ({i},{j-1})")
j -= 1

lcs.reverse()
return ".join(lcs)

def lcs_all_solutions(str1, str2):
    """Find all possible LCS solutions."""
    m, n = len(str1), len(str2)

    # Create DP table
dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Fill DP table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if str1[i-1] == str2[j-1]:
dp[i][j] = dp[i-1][j-1] + 1
            else:
dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    # Find all LCS strings using backtracking
all_lcs = set()

    def backtrack(i, j, current_lcs):
        if i == 0 or j == 0:
all_lcs.add(current_lcs[::-1]) # Reverse to get correct order
        return

        if str1[i-1] == str2[j-1]:
backtrack(i-1, j-1, current_lcs + str1[i-1])
        else:
            if dp[i-1][j] == dp[i][j-1]:
backtrack(i-1, j, current_lcs)

```

```

        if dp[i][j-1] == dp[i][j]:
            backtrack(i, j-1, current_lcs)

    backtrack(m, n, "")
    return list(all_lcs)

def demonstrate_lcs():
    """Demonstrate LCS computation."""
    print("=== Longest Common Subsequence Demonstration ===")

    test_pairs = [
        ("ABCDGH", "AEDFHR"),
        ("AGGTAB", "GXTXAYB"),
        ("PROGRAMMING", "CRAMMING"),
        ("ABCDEFGH", "ACEG")
    ]

    for str1, str2 in test_pairs:
        print(f"\n--- LCS: '{str1}' and '{str2}' ---")
        length = lcs_length(str1, str2)
        print(f"LCS length: {length}")

        lcs_str = lcs_string(str1, str2)
        print(f"LCS string: '{lcs_str}'")

        all_solutions = lcs_all_solutions(str1, str2)
        print(f"All LCS solutions: {all_solutions}")

    demonstrate_lcs()

```

13.5 Text Compression and the Greedy Method

Text compression reduces storage space and transmission time. Greedy algorithms like Huffman coding provide optimal prefix-free codes.

HUFFMAN CODING

Huffman coding assigns shorter codes to more frequent characters.

Huffman Coding Algorithm - High Level Steps

- **Step 1: Build Frequency Table**
 - Count the frequency of each character in the input text
 - Create a frequency table mapping each character to its occurrence count
- **Step 2: Create Initial Priority Queue**
 - Create a leaf node for each character with its frequency
 - Add all leaf nodes to a min-heap (priority queue) ordered by frequency
- **Step 3: Build Huffman Tree**
 - While there's more than one node in the heap:
 - Extract the two nodes with lowest frequencies
 - Create a new internal node with these as children
 - Set the internal node's frequency as the sum of its children's frequencies
 - Add the internal node back to the heap
 - The remaining node becomes the root of the Huffman tree
- **Step 4: Generate Codes**

- Traverse the tree from root to each leaf
- Assign '0' for left edges and '1' for right edges
- The path from root to leaf becomes that character's binary code
- More frequent characters end up with shorter codes
- **Step 5: Encode Text**
- Replace each character in the original text with its corresponding Huffman code
- Concatenate all codes to create the compressed binary string
- **Step 6: Decode Text (when needed)**
- Traverse the Huffman tree bit by bit using the encoded string
- When reaching a leaf node, output the character and return to root
- Continue until all bits are processed
- The algorithm uses a greedy approach to build an optimal prefix-free code where more frequent characters get shorter codes, resulting in overall compression.

Given below is the code implementation -

```
import heapq
from collections import defaultdict, Counter

class HuffmanNode:
    """Node class for Huffman tree."""

    def __init__(self, char=None, freq=0, left=None, right=None):
        self.char = char
        self.freq = freq
        self.left = left
        self.right = right

    def __lt__(self, other):
        """For heap comparison."""
        return self.freq < other.freq

    def __repr__(self):
        if self.char:
            return f'HuffmanNode('{self.char}': {self.freq})'
        else:
            return f'HuffmanNode(internal: {self.freq})'

class HuffmanCoding:
    """Huffman coding implementation for text compression."""

    def __init__(self):
        self.codes = {}
        self.reverse_codes = {}
        self.root = None

    def build_frequency_table(self, text):
        """Build frequency table for characters in text."""
        frequency = Counter(text)
        print(f'Character frequencies:')
        for char, freq in sorted(frequency.items()):
            print(f' {char}: {freq}')
```

```

        return frequency

    def build_huffman_tree(self, frequency):
        """Build Huffman tree using greedy algorithm."""
        if len(frequency) == 1:
            # Special case: only one unique character
            char = list(frequency.keys())[0]
            self.root = HuffmanNode(char, frequency[char])
            return self.root

        # Create priority queue with leaf nodes
        heap = []
        for char, freq in frequency.items():
            node = HuffmanNode(char, freq)
            heapq.heappush(heap, node)

        print(f"\nBuilding Huffman tree:")
        step = 1

        # Build tree bottom-up
        while len(heap) > 1:
            # Extract two nodes with minimum frequency
            left = heapq.heappop(heap)
            right = heapq.heappop(heap)

            # Create internal node
            internal_freq = left.freq + right.freq
            internal_node = HuffmanNode(None, internal_freq, left, right)

            print(f"Step {step}: Merge {left} + {right} = {internal_node}")
            step += 1

            # Add internal node back to heap
            heapq.heappush(heap, internal_node)

        self.root = heap[0]
        return self.root

    def generate_codes(self, node=None, code=""):
        """Generate Huffman codes for each character."""
        if node is None:
            node = self.root

        if node.char: # Leaf node
            if code == "": # Single character case
                code = "0"
            self.codes[node.char] = code
            self.reverse_codes[code] = node.char
            print(f" '{node.char}': {code}")
        else: # Internal node
            if node.left:
                self.generate_codes(node.left, code + "0")
            if node.right:
                self.generate_codes(node.right, code + "1")

```

```

def encode(self, text):
    """Encode text using Huffman codes."""
    if not self.codes:
        frequency = self.build_frequency_table(text)
    self.build_huffman_tree(frequency)
    print(f"\nGenerated Huffman codes:")
    self.generate_codes()

    encoded = ""
    for char in text:
        encoded += self.codes[char]

    return encoded

def decode(self, encoded_text):
    """Decode Huffman encoded text."""
    if not self.reverse_codes:
        raise ValueError("No codes available for decoding")

    decoded = ""
    current_code = ""

    for bit in encoded_text:
        current_code += bit
        if current_code in self.reverse_codes:
            decoded += self.reverse_codes[current_code]
        current_code = ""

    if current_code: # Incomplete code
        raise ValueError(f"Incomplete code: {current_code}")

    return decoded

def get_compression_stats(self, original_text, encoded_text):
    """Calculate compression statistics."""
    original_bits = len(original_text) * 8 # Assuming 8 bits per character
    compressed_bits = len(encoded_text)
    compression_ratio = compressed_bits / original_bits
    space_saved = 1 - compression_ratio

    return {
        'original_bits': original_bits,
        'compressed_bits': compressed_bits,
        'compression_ratio': compression_ratio,
        'space_saved': space_saved,
        'avg_bits_per_char': compressed_bits / len(original_text)
    }

def display_tree(self, node=None, level=0, prefix="Root: "):
    """Display Huffman tree structure."""
    if node is None:
        node = self.root

```

```

        if node:
            if node.char:
                print(" " * (level * 4) + prefix + f"{node.char}' (freq: {node.freq})")
            else:
                print(" " * (level * 4) + prefix + f"Internal (freq: {node.freq})")
                if node.left or node.right:
                    self.display_tree(node.left, level + 1, "L--- ")
                    self.display_tree(node.right, level + 1, "R--- ")

def demonstrate_huffman_coding():
    """Demonstrate Huffman coding algorithm."""
    print("=== Huffman Coding Demonstration ===")

    # Test text with varying character frequencies
    text = "ABRACADABRA"

    print(f"Original text: '{text}'")
    print(f"Length: {len(text)} characters")

    # Create Huffman coder and encode
    huffman = HuffmanCoding()
    encoded = huffman.encode(text)

    print(f"\nHuffman tree structure:")
    huffman.display_tree()

    print(f"\nEncoded text: {encoded}")
    print(f"Encoded length: {len(encoded)} bits")

    # Decode and verify
    decoded = huffman.decode(encoded)
    print(f"Decoded text: '{decoded}'")
    print(f"Decoding successful: {decoded == text}")

    # Compression statistics
    stats = huffman.get_compression_stats(text, encoded)
    print(f"\nCompression Statistics:")
    print(f" Original size: {stats['original_bits']} bits")
    print(f" Compressed size: {stats['compressed_bits']} bits")
    print(f" Compression ratio: {stats['compression_ratio']:.3f}")
    print(f" Space saved: {stats['space_saved']:.1%}")
    print(f" Average bits per character: {stats['avg_bits_per_char']:.2f}")

demonstrate_huffman_coding()

```

RUN-LENGTH ENCODING

Simple compression for data with many consecutive repeated values

Run-Length Encoding Algorithm - High Level Steps

Encoding Process

Step 1: Initialize Variables

- Start with the first character as the current character
- Set count to 1
- Create an empty result list

Step 2: Scan Through Text

- For each subsequent character in the input:
 - If it matches the current character: increment the count
 - If it's different: record the current character and its count, then reset for the new character

Step 3: Record Final Run

- After scanning all characters, don't forget to record the last character and its count

Step 4: Format Output

- Combine each character with its count (e.g., "A3B2C1")
- Join all runs into the final encoded string

Decoding Process

Step 1: Initialize Parsing

- Start at the beginning of the encoded string
- Create an empty result list

Step 2: Parse Character-Count Pairs

- Read each character
- Read the following digits to get the count (handle multi-digit numbers)
- Repeat the character that many times

Step 3: Reconstruct Text

- Concatenate all repeated character sequences
- Return the original text

Key Characteristics

- **Best for:** Data with many consecutive repeated characters
- **Compression efficiency:** Depends on repetition patterns - can actually increase size if there are few repeats
- **Simple and fast:** Linear time complexity $O(n)$
- **Lossless:** Perfect reconstruction of original data

The algorithm works by replacing runs of identical consecutive characters with a single character followed by the count of repetitions.

Sample implementation is given below -

```
class RunLengthEncoding:
    """Run-Length Encoding implementation."""

    @staticmethod
    def encode(text):
        """
        Encode text using run-length encoding.
        Format: character followed by count
        """
        if not text:
            return ""

        encoded = []
        current_char = text[0]
        count = 1

        print(f'Encoding text: '{text}')
        print("Step-by-step encoding:")

        for i in range(1, len(text)):
            if text[i] == current_char:
                count += 1
```

```

print(f' Position {i}: '{text[i]}' matches, count = {count}')
    else:
        # Different character found, record current run
        encoded.append(f'{current_char} {count}')
        print(f' Position {i}: '{text[i]}' differs, record '{current_char} {count}''')
        current_char = text[i]
        count = 1

    # Don't forget the last run
    encoded.append(f'{current_char} {count}')
    print(f' End: record final run '{current_char} {count}''')

    result = ".join(encoded)
    print(f'Encoded result: '{result}''')
    return result

    @staticmethod
    def decode(encoded_text):
        """
        Decode run-length encoded text.
        """
        if not encoded_text:
            return ""

        decoded = []
        i = 0

        print(f'Decoding text: '{encoded_text}''')
        print("Step-by-step decoding:")

        while i < len(encoded_text):
            char = encoded_text[i]
            i += 1

            # Read count (may be multiple digits)
            count_str = ""
            while i < len(encoded_text) and encoded_text[i].isdigit():
                count_str += encoded_text[i]
                i += 1

            count = int(count_str)
            decoded_part = char * count
            decoded.append(decoded_part)

        print(f' '{char}' × {count} = '{decoded_part}''')

        result = ".join(decoded)
        print(f'Decoded result: '{result}''')
        return result

    def demonstrate_rle():
        """Demonstrate Run-Length Encoding."""
        print("=== Run-Length Encoding Demonstration ===")

```

```

test_strings = [
    "AAABBBCCDAAA",

    "WWWWWWWWWWWWBWWWWWWWWWWWWBBBWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWWW",
    "ABCDEF", # No repetition
    "AAAAAAAAAA" # All same character
]

rle = RunLengthEncoding()

for text in test_strings:
    print(f'\n--- Test String: '{text}' ---')
    print(f'Original length: {len(text)}')

    # Encode
    encoded = rle.encode(text)
    print(f'Encoded length: {len(encoded)}')

    # Calculate compression ratio
    if len(text) > 0:
        ratio = len(encoded) / len(text)
    print(f'Compression ratio: {ratio:.2f}')

    # Decode and verify
    decoded = rle.decode(encoded)
    print(f'Decoding successful: {decoded == text}')

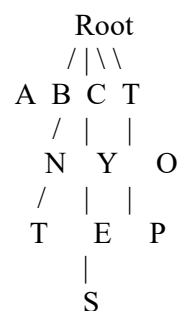
demonstrate_rle()

```

13.6 Tries

A trie (prefix tree) is a specialized tree data structure for storing strings, enabling efficient prefix-based operations.

Trie Structure Visualization:



Words: "ANT", "BYE", "BYES", "TOP"

Trie Data Structure - High Level Steps

Insert Operation

Step 1: Start at Root

- Begin traversal from the root node of the trie

Step 2: Traverse Character by Character

- For each character in the word:
 - If the character exists as a child: move to that child node
 - If the character doesn't exist: create a new child node and move to it

Step 3: Mark End of Word

- At the final character, mark the node as "end of word"
- Increment word count and trie size

Search Operation**Step 1: Start at Root**

- Begin traversal from the root node

Step 2: Follow Character Path

- For each character in the target word:
 - If character exists as a child: move to that child node
 - If character doesn't exist: return false (word not found)

Step 3: Verify Complete Word

- After traversing all characters, check if current node is marked as "end of word"
- Return true only if it's a complete word, not just a prefix

Prefix Search Operation**Step 1: Navigate to Prefix End**

- Follow the same path as search, but stop after the prefix characters

Step 2: Check Existence

- If all prefix characters found: return true
- If any character missing: return false

Delete Operation**Step 1: Verify Word Exists**

- First search to confirm the word exists in the trie

Step 2: Recursive Deletion

- Traverse to the end of the word
- Unmark "end of word" flag
- Work backwards, deleting nodes that have no other children and aren't end of other words

Step 3: Clean Up

- Remove unnecessary nodes to keep trie structure minimal

Get Words with Prefix**Step 1: Navigate to Prefix Node**

- Traverse to the node representing the end of the prefix

Step 2: Collect All Descendants

- Recursively traverse all paths from that node
- Build complete words by concatenating characters along each path
- Add words to result list when reaching "end of word" nodes

Key Characteristics

- **Time Complexity:** $O(m)$ for most operations, where m is word length
- **Space Efficient:** Shared prefixes use same nodes
- **Fast Prefix Operations:** Excellent for autocomplete and prefix matching
- **Ordered Storage:** Can retrieve words in lexicographical order

The trie efficiently stores strings by sharing common prefixes, making it ideal for applications like spell checkers, autocomplete systems, and IP routing tables.

The sample code is given below –

```
class TrieNode:
```

```

"""Node class for Trie data structure."""

def __init__(self):
    """Initialize trie node."""
self.children = {} # Dictionary mapping characters to child nodes
self.is_end_of_word = False # Marks end of a valid word
self.word_count = 0 # Number of words ending at this node

def __repr__(self):
    return f'TrieNode(children={list(self.children.keys())},
is_end={self.is_end_of_word})'

class Trie:
    """Trie (Prefix Tree) implementation for string storage and retrieval."""

    def __init__(self):
        """Initialize empty trie."""
self.root = TrieNode()
self.size = 0 # Total number of words

    def insert(self, word):
        """
        Insert a word into the trie.
        Time Complexity: O(m) where m is length of word
        """
        if not word:
            return

        current = self.root
        print(f'Inserting word: '{word}''')

        for i, char in enumerate(word):
            if char not in current.children:
                current.children[char] = TrieNode()
                print(f' Created new node for '{char}' at level {i+1}''')
            else:
                print(f' Found existing node for '{char}' at level {i+1}''')

            current = current.children[char]

        if not current.is_end_of_word:
            current.is_end_of_word = True
            self.size += 1
            print(f' Marked end of word at '{word}''')
        else:
            print(f' Word '{word}' already exists''')

        current.word_count += 1

    def search(self, word):
        """
        Search for a word in the trie.
        Time Complexity: O(m) where m is length of word
        """

```

```

        current = self.root
    print(f'Searching for word: '{word}''')

    for i, char in enumerate(word):
        if char not in current.children:
            print(f' Character '{char}' not found at level {i+1}''')
            return False
        print(f' Found '{char}' at level {i+1}''')
        current = current.children[char]

    found = current.is_end_of_word
    print(f' Word found: {found}''')
    return found

def starts_with(self, prefix):
    """
    Check if any word in trie starts with given prefix.
    Time Complexity: O(m) where m is length of prefix
    """
    current = self.root
    print(f'Checking prefix: '{prefix}''')

    for i, char in enumerate(prefix):
        if char not in current.children:
            print(f' Prefix not found: '{char}' missing at level {i+1}''')
            return False
        print(f' Found '{char}' at level {i+1}''')
        current = current.children[char]

    print(f' Prefix '{prefix}' exists''')
    return True

def delete(self, word):
    """
    Delete a word from the trie.
    Time Complexity: O(m) where m is length of word
    """
    if not self.search(word):
        print(f'Word '{word}' not found for deletion''')
        return False

    def _delete_helper(node, word, index):
        if index == len(word):
            # Reached end of word
            if not node.is_end_of_word:
                return False # Word doesn't exist

        node.is_end_of_word = False
        node.word_count -= 1
        # Return True if node has no children (can be deleted)
        return len(node.children) == 0

    char = word[index]
    child_node = node.children.get(char)

```

```

        if not child_node:
            return False # Word doesn't exist

should_delete_child = _delete_helper(child_node, word, index + 1)

        if should_delete_child:
            del node.children[char]
            # Return True if current node can also be deleted
            return (not node.is_end_of_word and
len(node.children) == 0)

        return False

print(f'Deleting word: '{word}''')
_delete_helper(self.root, word, 0)
self.size -= 1
print(f'Word '{word}' deleted successfully')
return True

def get_all_words_with_prefix(self, prefix):
    """Get all words that start with given prefix."""
    current = self.root

    # Navigate to prefix node
    for char in prefix:
        if char not in current.children:
            return [] # Prefix doesn't exist
        current = current.children[char]

    # Collect all words from this node
    words = []
self._collect_words(current, prefix, words)
    return words

def _collect_words(self, node, current_word, words):
    """Recursively collect all words from a node."""
    if node.is_end_of_word:
words.append(current_word)

        for char, child_node in node.children.items():
self._collect_words(child_node, current_word + char, words)

def get_all_words(self):
    """Get all words stored in the trie."""
    words = []
self._collect_words(self.root, "", words)
    return sorted(words)

def count_words_with_prefix(self, prefix):
    """Count words that start with given prefix."""
    current = self.root

    # Navigate to prefix node

```

```

    for char in prefix:
        if char not in current.children:
            return 0
        current = current.children[char]

    return self._count_words(current)

def _count_words(self, node):
    """Count all words from a given node."""
    count = 1 if node.is_end_of_word else 0

    for child_node in node.children.values():
        count += self._count_words(child_node)

    return count

def longest_common_prefix(self):
    """Find the longest common prefix of all words in trie."""
    if self.size == 0:
        return ""

    current = self.root
    prefix = ""

    while (len(current.children) == 1 and
           not current.is_end_of_word):
        char = list(current.children.keys())[0]
        prefix += char
        current = current.children[char]

    return prefix

def display_structure(self, node=None, level=0, char="", prefix=""):
    """Display trie structure for visualization."""
    if node is None:
        node = self.root
    print("Trie Structure:")

    # Show current node
    if char:
        indicator = " (END)" if node.is_end_of_word else ""
        print(" " * level + f" |—— '{char}'>{indicator}")
    else:
        print(" " * level + "Root")

    # Show children
    children = list(node.children.items())
    for i, (child_char, child_node) in enumerate(children):
        is_last = (i == len(children) - 1)
        child_prefix = prefix + char
        self.display_structure(child_node, level + 1, child_char, child_prefix)

def demonstrate_trie():
    """Demonstrate Trie data structure operations."""

```

```

print("=== Trie Data Structure Demonstration ===")

# Create trie and insert words
trie = Trie()
words = ["CAT", "CATS", "DOG", "DOGS", "DOGGY", "CAR", "CARD",
"CARE", "CAREFUL"]

print("--- Inserting Words ---")
for word in words:
    trie.insert(word)
print()

print(f'Total words in trie: {trie.size}')

# Display trie structure
print("\n--- Trie Structure ---")
trie.display_structure()

# Search operations
print("\n--- Search Operations ---")
search_words = ["CAT", "CATCH", "DOG", "CARE"]
for word in search_words:
    found = trie.search(word)
    print(f'Search '{word}': {found}')
print()

# Prefix operations
print("--- Prefix Operations ---")
prefixes = ["CA", "DO", "XY"]
for prefix in prefixes:
    has_prefix = trie.starts_with(prefix)
    print(f'Has prefix '{prefix}': {has_prefix}')

    if has_prefix:
        words_with_prefix = trie.get_all_words_with_prefix(prefix)
        print(f'Words with prefix '{prefix}': {words_with_prefix}')
        print()

# Get all words
print("--- All Words in Trie ---")
all_words = trie.get_all_words()
print(f'All words: {all_words}')

# Longest common prefix
print(f'\nLongest common prefix: '{trie.longest_common_prefix()}'")

# Delete operations
print("\n--- Delete Operations ---")
delete_words = ["CATS", "DOGGY"]
for word in delete_words:
    trie.delete(word)
print(f'Remaining words: {trie.get_all_words()}')
print()

```

COMPRESSED TRIE (PATRICIA TREE)

A compressed trie reduces space by merging nodes with single children:

```
class CompressedTrieNode:
    """Node for compressed trie (PATRICIA tree)."""

    def __init__(self, label=""):
        """Initialize with edge label."""
    self.label = label # String label on edge to this node
    self.children = {}
    self.is_end_of_word = False
    self.words = [] # Store actual words ending at this node

    def __repr__(self):
        return f'CompressedTrieNode(label='{self.label}',
end='{self.is_end_of_word}')"

class CompressedTrie:
    """Compressed Trie (PATRICIA tree) implementation."""

    def __init__(self):
        """Initialize empty compressed trie."""
    self.root = CompressedTrieNode()
    self.size = 0

    def insert(self, word):
        """Insert word into compressed trie."""
        if not word:
            return

    print(f'Inserting word: '{word}''')
        current = self.root
        remaining = word

        while remaining:
            # Find matching child
            matching_child = None
            for child in current.children.values():
                if remaining.startswith(child.label[0]):
                    matching_child = child
                    break

            if matching_child is None:
                # No matching child, create new node
                new_node = CompressedTrieNode(remaining)
                new_node.is_end_of_word = True
                new_node.words.append(word)
                current.children[remaining[0]] = new_node
                print(f' Created new node with label '{remaining}''')
                break
            else:
                # Found matching child
```

```

child_label = matching_child.label
common_prefix = self._find_common_prefix(remaining, child_label)

    if common_prefix == child_label:
        # Child label is prefix of remaining string
    print(f" Traversing node with label '{child_label}'")
        current = matching_child
        remaining = remaining[len(common_prefix):]

    if not remaining:
        # Word ends at this node
        if not matching_child.is_end_of_word:
            matching_child.is_end_of_word = True
            self.size += 1
            matching_child.words.append(word)
            print(f" Word ends at existing node")

    elif common_prefix == remaining:
        # Remaining string is prefix of child label
        # Need to split the child node
    print(f" Splitting node: common='{common_prefix}', child='{child_label}'")

        # Create new intermediate node
        intermediate = CompressedTrieNode(common_prefix)
        intermediate.is_end_of_word = True
        intermediate.words.append(word)

        # Update existing child
        matching_child.label = child_label[len(common_prefix):]

        # Reconnect
        del current.children[child_label[0]]
        current.children[common_prefix[0]] = intermediate
        intermediate.children[matching_child.label[0]] = matching_child

    self.size += 1
    break

    else:
        # Partial match - need to split
    print(f" Partial match: common='{common_prefix}'")

        # Create intermediate node
        intermediate = CompressedTrieNode(common_prefix)

        # Create new leaf for remaining word
        remaining_suffix = remaining[len(common_prefix):]
        new_leaf = CompressedTrieNode(remaining_suffix)
        new_leaf.is_end_of_word = True
        new_leaf.words.append(word)

        # Update existing child
        matching_child.label = child_label[len(common_prefix):]

```

```

        # Reconnect all nodes
        del current.children[child_label[0]]
current.children[common_prefix[0]] = intermediate
intermediate.children[remaining_suffix[0]] = new_leaf
intermediate.children[matching_child.label[0]] = matching_child

self.size += 1
        break

    def _find_common_prefix(self, str1, str2):
        """Find common prefix of two strings."""
i = 0
        while (i < len(str1) and i < len(str2) and str1[i] == str2[i]):
i += 1
        return str1[:i]

    def search(self, word):
        """Search for word in compressed trie."""
        current = self.root
        remaining = word

print(f"Searching for: '{word}'")

        while remaining and current:
found_child = False

            for child in current.children.values():
                if remaining.startswith(child.label):
print(f" Matched label '{child.label}'")
                    current = child
                    remaining = remaining[len(child.label):]
found_child = True
                    break

            if not found_child:
print(f" No matching child for '{remaining}'")
                return False

        result = not remaining and current and current.is_end_of_word
print(f" Search result: {result}")
        return result

    def display_structure(self, node=None, level=0, edge_label=""):
        """Display compressed trie structure."""
        if node is None:
            node = self.root
print("Compressed Trie Structure:")
            print("Root")
        else:
end_marker = " (END)" if node.is_end_of_word else ""
words_info = f" {node.words}" if node.words else ""
print(" " * level + f" |—— '{edge_label}'{end_marker} {words_info}")

        for child in node.children.values():

```

```

self.display_structure(child, level + 1, child.label)

def demonstrate_compressed_trie():
    """Demonstrate Compressed Trie operations."""
    print("=== Compressed Trie (PATRICIA) Demonstration ===")

    # Create compressed trie
    ctrie = CompressedTrie()

    # Insert words that will demonstrate compression
    words = ["CAR", "CARD", "CARE", "CAREFUL", "CAT", "CATCH"]

    print("--- Inserting Words ---")
    for word in words:
        ctrie.insert(word)
    print()

    print(f"Total words: {ctrie.size}")

    # Display structure
    print("\n--- Compressed Trie Structure ---")
    ctrie.display_structure()

    # Search operations
    print("\n--- Search Operations ---")
    search_words = ["CAR", "CARD", "CAREER", "CAT"]
    for word in search_words:
        ctrie.search(word)
    print()

    demonstrate_compressed_trie()

```

Check Your Progress 1

- 1. Question:** What is the time complexity of the brute force pattern matching algorithm? a) $O(n + m)$ b) $O(n \times m)$ c) $O(n \log m)$ d) $O(m \log n)$
- 2. Question:** What advantage does the KMP algorithm provide over brute force pattern matching? a) Uses less memory b) Avoids redundant character comparisons c) Works only with binary strings d) Requires preprocessing of the text
- 3. Question:** In dynamic programming for edit distance, what do the three operations represent? a) Insert, delete, copy b) Insert, delete, substitute c) Match, substitute, transpose d) Copy, move, replace
- 4. Question:** What is the main principle behind Huffman coding? a) Assign equal-length codes to all characters b) Assign shorter codes to more frequent characters c) Use alphabetical ordering for code assignment d) Minimize the number of unique characters
- 5. Question:** What is the time complexity of searching for a word of length m in a trie? a) $O(\log m)$ b) $O(m \log n)$ c) $O(m)$ d) $O(n)$

6. Question: Which compression technique is most suitable for data with many consecutive repeated characters? a) Huffman coding b) Run-length encoding c) LZ77 d) Arithmetic coding

7. Question: What is the space complexity of computing edit distance using dynamic programming? a) $O(1)$ b) $O(m)$ c) $O(n)$ d) $O(m \times n)$

8. Question: In a compressed trie (PATRICIA tree), what happens to nodes with only one child? a) They are deleted b) They are merged with their parent c) They are merged with their child d) They remain unchanged

9. Question: What is the main advantage of the Boyer-Moore algorithm? a) It works from left to right b) It can skip multiple characters when a mismatch occurs c) It uses constant extra space d) It works only with ASCII characters

10. Question: Which property makes Huffman codes uniquely decodable? a) All codes have the same length b) Codes are assigned alphabetically c) No code is a prefix of another code d) Codes use only 0s and 1s

13.7 Summary

Text processing algorithms and data structures are fundamental to modern computing, enabling efficient handling of the massive amounts of digital text data generated daily. This unit covered essential concepts and techniques that form the backbone of search engines, compression tools, bioinformatics applications, and natural language processing systems.

Key Concepts Covered:

- **Digitized Text Challenges:** Understanding the scale and complexity of modern text processing, including encoding issues, memory constraints, and the need for efficient algorithms that can handle terabytes of data.
- **Pattern-Matching Algorithms:** We explored multiple approaches from the simple $O(nm)$ brute force method to sophisticated algorithms like KMP $O(n+m)$, Boyer-Moore, and Rabin-Karp that use preprocessing and heuristics to achieve better average-case performance.
- **Dynamic Programming in Text Processing:** Applied DP techniques to solve fundamental problems like edit distance calculation, longest common subsequence finding, and sequence alignment - crucial for bioinformatics and spell checking applications.
- **Text Compression:** Implemented greedy algorithms including Huffman coding for optimal prefix-free encoding, run-length encoding for repetitive data, and dictionary-based methods like LZ77 that achieve significant space savings.
- **Trie Data Structures:** Explored prefix trees and compressed tries (PATRICIA trees) that enable efficient string storage, fast prefix matching, and applications like autocomplete systems and spell checkers.

Performance Summary:

- Pattern matching: $O(n+m)$ with KMP, $O(n/m)$ average with Boyer-Moore
- Edit distance: $O(mn)$ time and space complexity
- Huffman coding: $O(n \log n)$ for tree construction, $O(n)$ for encoding/decoding
- Trie operations: $O(m)$ where m is string length

- Space-time tradeoffs: Tries use more space but provide faster lookup than hash tables for prefix operations

Real-World Applications:

- Search engines use pattern matching for query processing and indexing
- Compression algorithms reduce storage and transmission costs
- Bioinformatics relies on sequence alignment for DNA/protein analysis
- Autocomplete and spell-check systems use tries for fast prefix matching
- Version control systems use text differencing algorithms

These algorithms and data structures provide the foundation for advanced text processing applications and continue to be relevant as the volume of digital text grows exponentially.

13.8 Solutions/Answers to Check Your Progress

1. Question: What is the time complexity of the brute force pattern matching algorithm? a) $O(n + m)$ b) $O(n \times m)$ c) $O(n \log m)$ d) $O(m \log n)$

Answer: b) $O(n \times m)$

Explanation: The brute force algorithm checks every position in the text (n positions) and for each position, compares up to m characters of the pattern. In the worst case, this results in $O(n \times m)$ time complexity.

2. Question: What advantage does the KMP algorithm provide over brute force pattern matching? a) Uses less memory b) Avoids redundant character comparisons c) Works only with binary strings d) Requires preprocessing of the text

Answer: b) Avoids redundant character comparisons

Explanation: KMP algorithm preprocesses the pattern to create a failure function (LPS array) that allows it to skip redundant comparisons when a mismatch occurs, achieving $O(n + m)$ time complexity instead of $O(n \times m)$.

3. Question: In dynamic programming for edit distance, what do the three operations represent? a) Insert, delete, copy b) Insert, delete, substitute c) Match, substitute, transpose d) Copy, move, replace

Answer: b) Insert, delete, substitute

Explanation: The edit distance (Levenshtein distance) considers three basic operations: inserting a character, deleting a character, and substituting one character for another.

4. Question: What is the main principle behind Huffman coding? a) Assign equal-length codes to all characters b) Assign shorter codes to more frequent characters c) Use alphabetical ordering for code assignment d) Minimize the number of unique characters

Answer: b) Assign shorter codes to more frequent characters

Explanation: Huffman coding is a greedy algorithm that builds an optimal prefix-free code by assigning shorter binary codes to characters that appear more frequently in the text.

5. Question: What is the time complexity of searching for a word of length m in a trie? a) $O(\log m)$ b) $O(m \log n)$ c) $O(m)$ d) $O(n)$

Answer: c) $O(m)$

Explanation: Searching in a trie requires traversing from root to leaf following the characters of the word. Since we visit exactly m nodes (one for each character), the time complexity is $O(m)$.

6. Question: Which compression technique is most suitable for data with many consecutive repeated characters? a) Huffman coding b) Run-length encoding c) LZ77 d) Arithmetic coding

Answer: b) Run-length encoding

Explanation: Run-length encoding is specifically designed for data with consecutive repeated characters, representing them as character-count pairs (e.g., "AAAAA" becomes "A5").

7. Question: What is the space complexity of computing edit distance using dynamic programming? a) $O(1)$ b) $O(m)$ c) $O(n)$ d) $O(m \times n)$

Answer: d) $O(m \times n)$

Explanation: The standard DP approach for edit distance creates a 2D table of size $(m+1) \times (n+1)$ to store intermediate results, resulting in $O(m \times n)$ space complexity.

8. Question: In a compressed trie (PATRICIA tree), what happens to nodes with only one child? a) They are deleted b) They are merged with their parent c) They are merged with their child d) They remain unchanged

Answer: c) They are merged with their child

Explanation: Compressed tries reduce space by merging nodes that have only one child with their child node, storing the edge labels to maintain the original string structure.

9. Question: What is the main advantage of the Boyer-Moore algorithm? a) It works from left to right b) It can skip multiple characters when a mismatch occurs c) It uses constant extra space d) It works only with ASCII characters

Answer: b) It can skip multiple characters when a mismatch occurs

Explanation: Boyer-Moore algorithm uses the bad character rule and good suffix rule to skip multiple positions when a mismatch occurs, potentially achieving $O(n/m)$ time complexity in the best case.

10. Question: Which property makes Huffman codes uniquely decodable? a) All codes have the same length b) Codes are assigned alphabetically c) No code is a prefix of another code d) Codes use only 0s and 1s

Answer: c) No code is a prefix of another code

Explanation: Huffman codes are prefix-free, meaning no code word is a prefix of another. This property ensures that the encoded text can be uniquely decoded without ambiguity.

13.9 Further Readings

- "Algorithms on Strings, Trees, and Sequences" by Dan Gusfield
- "Introduction to Algorithms" by Cormen, Leiserson, Rivest, and Stein
- "Pattern Recognition and Machine Learning" by Christopher Bishop
- "Compilers: Principles, Techniques, and Tools" by Aho, Sethi, and Ullman

Online Resources:

- Stanford CS97SI: Introduction to Computational Biology
- MIT 6.046J: Design and Analysis of Algorithms

- Coursera: Algorithms on Strings Specialization
- GeeksforGeeks: String Algorithms and Data Structures

Linked List

Research Papers:

- "A Fast String Searching Algorithm" by Boyer and Moore (1977)
- "Fast Pattern Matching in Strings" by Knuth, Morris, and Pratt (1977)
- "A Method for the Construction of Minimum-Redundancy Codes" by Huffman (1952)
- "A Universal Algorithm for Sequential Data Compression" by Ziv and Lempel (1977)

Software Tools:

- BLAST for biological sequence alignment
- Apache Lucene for text search and indexing
- Various compression utilities (gzip, bzip2, 7zip)
- Text editors with advanced search capabilities

